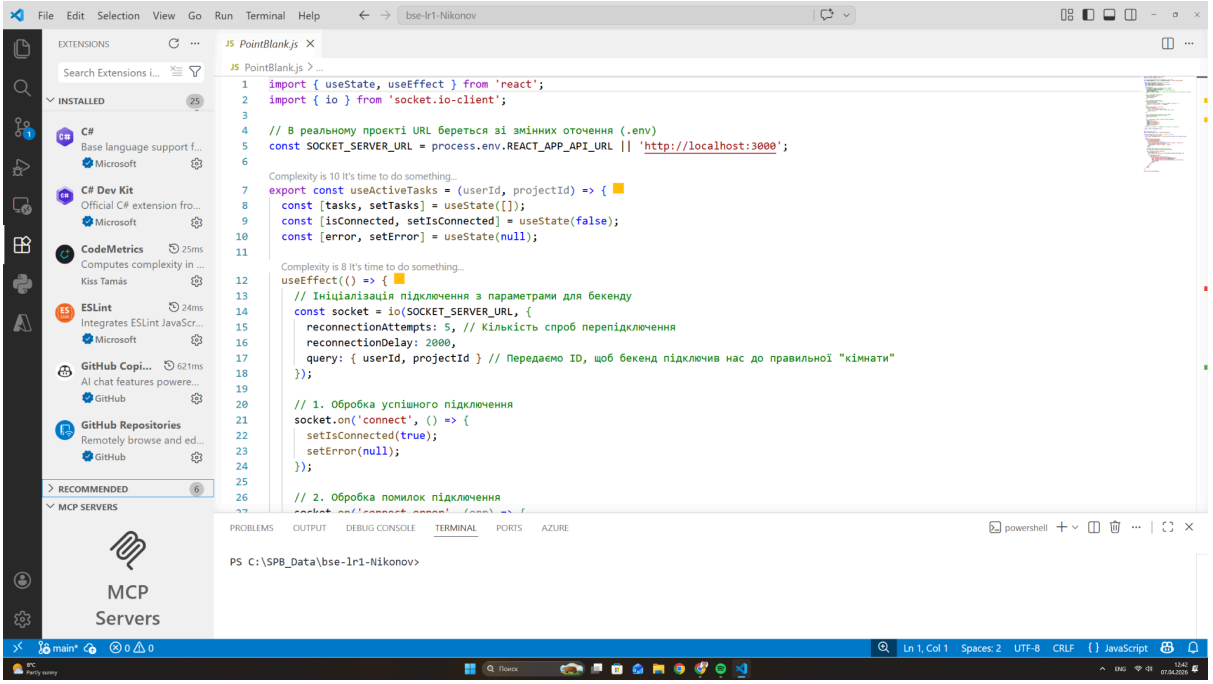
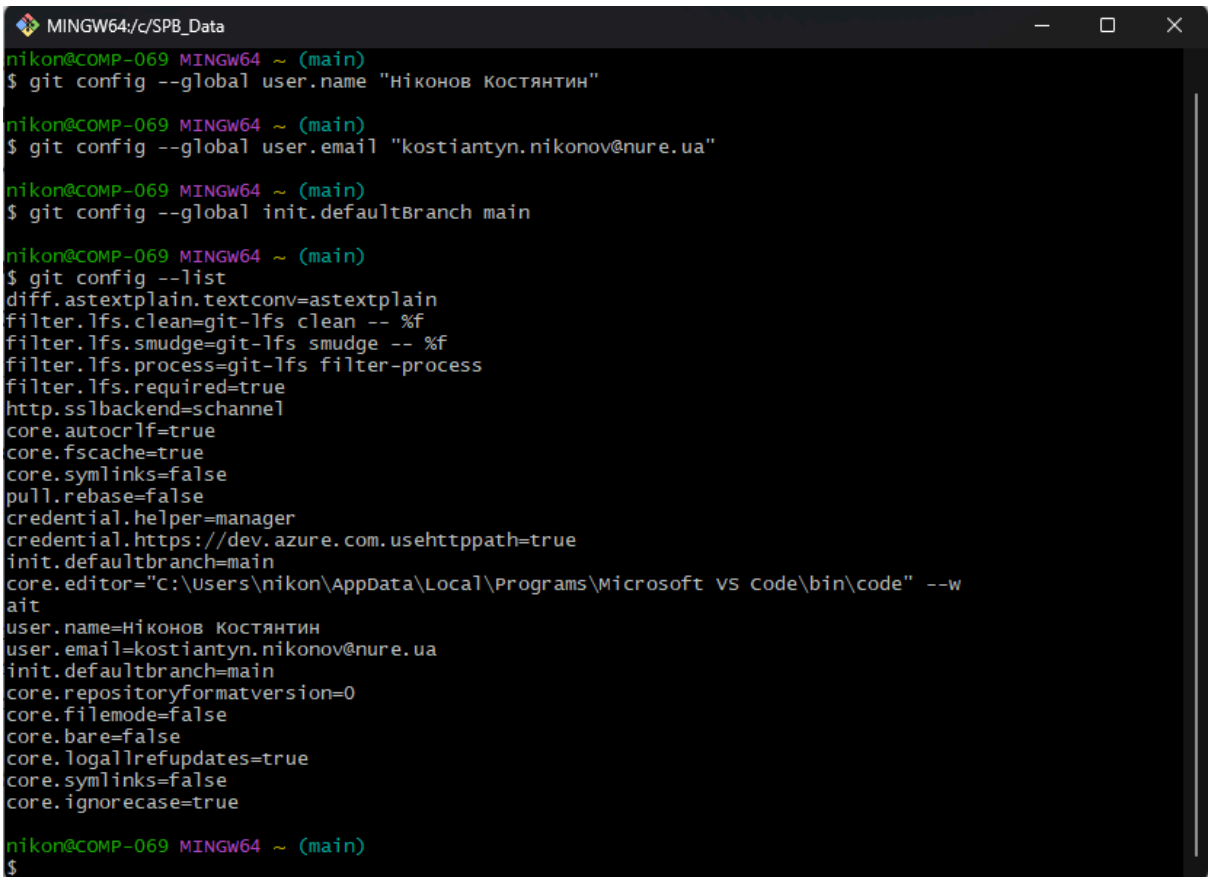
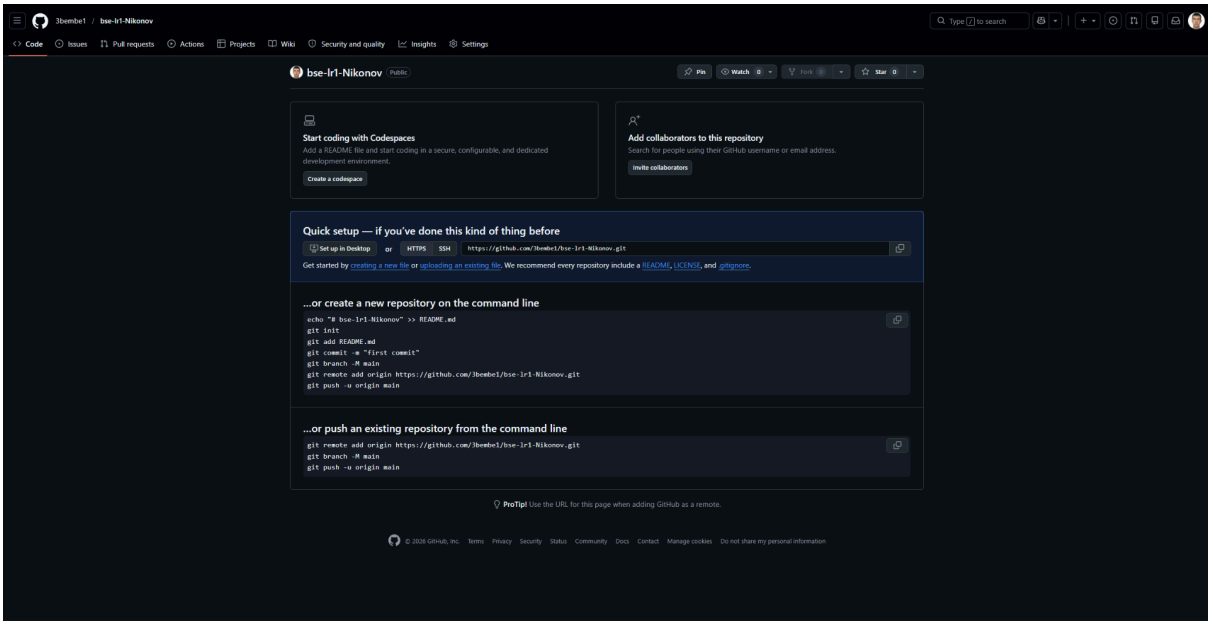
вебпокликання на GitHub: https://github.com/3bembe1/bse-lr1-Nikonov.git



1. Налаштоване робоче середовище VS Code

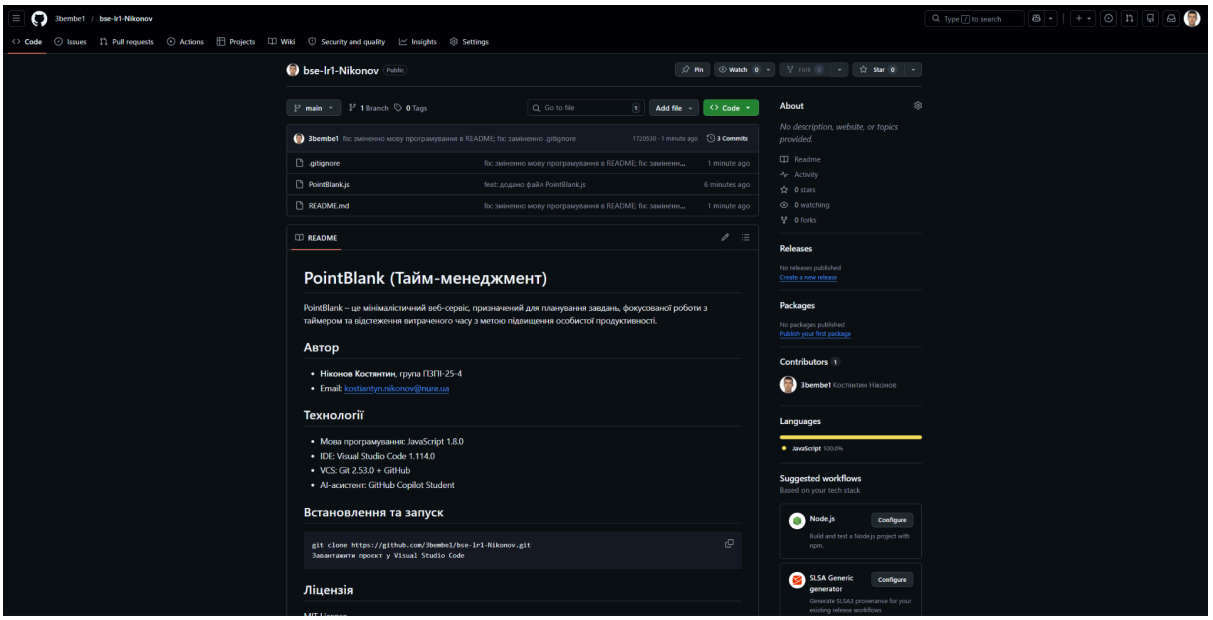


2. Перевірка конфігурації: git config --list

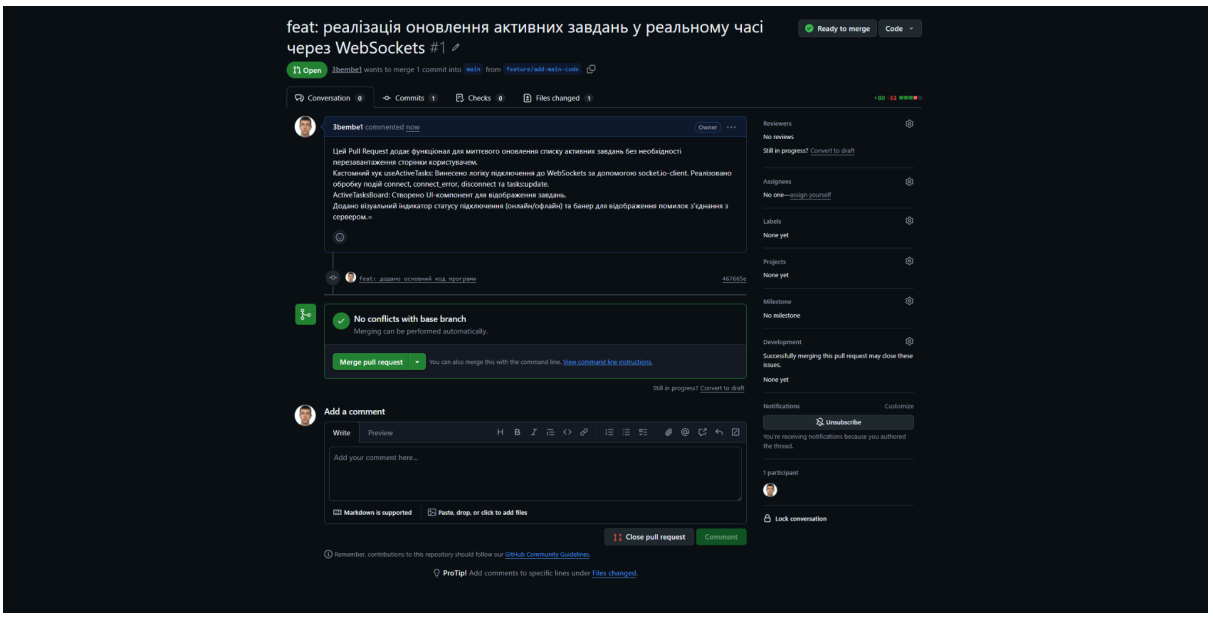


3. Створений репозиторій на GitHub

```
nikon@COMP-069 MINGW64 ~/bse-lr1-Nikonov (main)
$ git add .

nikon@COMP-069 MINGW64 ~/bse-lr1-Nikonov (main)
$ git commit -m "fix: змінено мову програмування в README; fix: замінено .gitignore"
[main 1720530] fix: змінено мову програмування в README; fix: замінено .gitignore
 2 files changed, 88 insertions(+), 132 deletions(-)

nikon@COMP-069 MINGW64 ~/bse-lr1-Nikonov (main)
$ git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 24 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 1.07 KiB | 1.07 MiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/3bembe1/bse-lr1-Nikonov.git
   1264974..1720530  main -> main

nikon@COMP-069 MINGW64 ~/bse-lr1-Nikonov (main)
$ git log --oneline --graph
* 1720530 (HEAD -> main, origin/main) fix: змінено мову програмування в README; fix: замінено .giti
gnore
* 1264974 feat: додано файл PointBlank.js
* effc752 Initial commit: додано README.md та .gitignore

nikon@COMP-069 MINGW64 ~/bse-lr1-Nikonov (main)
$ |
```



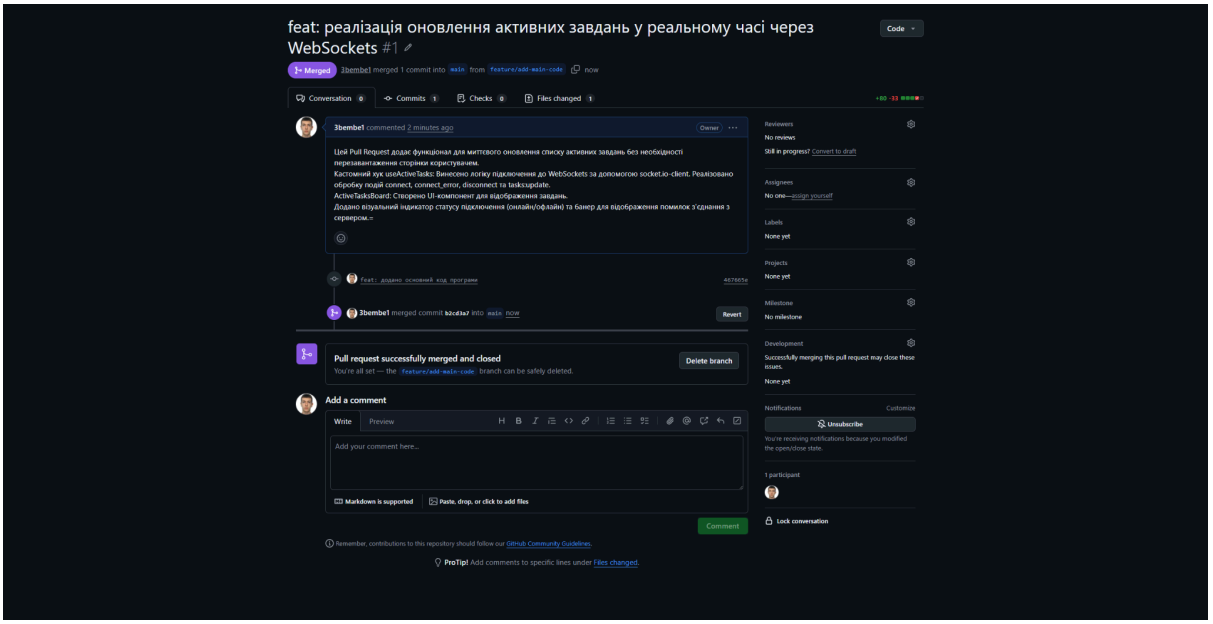
4. git log --oneline --graph та сторінка репозиторію на GitHub.



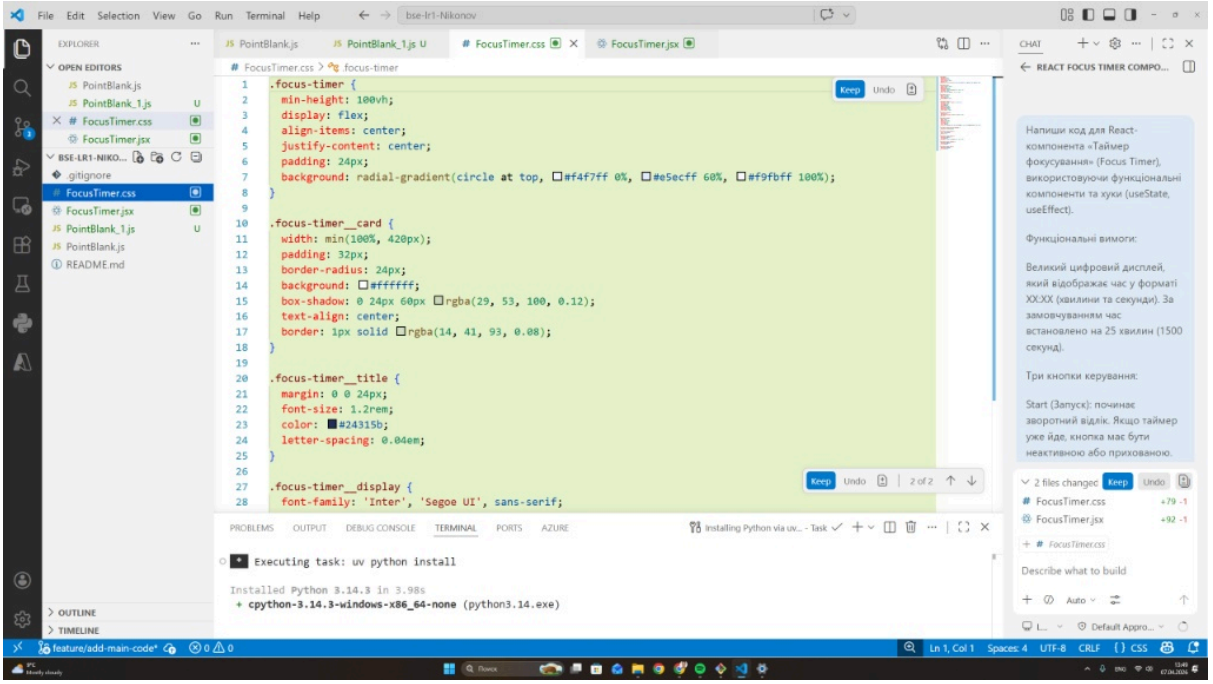

```diff
@@ -1,46 +1,93 @@
- import React, { useState, useEffect } from 'react';
+ import { useState, useEffect } from 'react';
+ import { io } from 'socket.io-client';

- const ActiveTasksList = () => {
+ const SOCKET_SERVER_URL = process.env.REACT_APP_API_URL || 'http://localhost:3000';
+
+ export const useActiveTasks = (userId, projectId) => {
  const [tasks, setTasks] = useState([]);
+  const [isConnected, setIsConnected] = useState(false);
+  const [error, setError] = useState(null);
+
+  useEffect(() => {
+    const socket = io(SOCKET_SERVER_URL, {
+      reconnectionAttempts: 5,
+      reconnectionDelay: 2000,
+      query: { userId, projectId }
+    });

-  const fetchActiveTasks = async () => {
-    try {
-      const response = await fetch('/api/tasks/active');
-      if (response.ok) {
-        const data = await response.json();
-        setTasks(data);
+    socket.on('connect', () => {
+      setIsConnected(true);
+      setError(null);
+    });
+
+    socket.on('connect_error', (err) => {
+      setIsConnected(false);
+      setError('Втрачено з\'єднання з сервером. Намагаємось відновити...');
+      console.error('Socket error:', err.message);
+    });
+
+    socket.on('disconnect', (reason) => {
+      setIsConnected(false);
+      if (reason === 'io server disconnect') {
+        socket.connect();
      }
-    } catch (error) {
-      console.error('Помилка при оновленні завдань:', error);
-    }
```

```diff
-  };
+    });

-  useEffect(() => {
-    fetchActiveTasks();
+    socket.on('tasks:update', (updatedTasks) => {
+      setTasks(updatedTasks);
+    });

-    const intervalId = setInterval(() => {
-      fetchActiveTasks();
-    }, 5000);
+    return () => {
+      socket.off('connect');
+      socket.off('connect_error');
+      socket.off('disconnect');
+      socket.off('tasks:update');
+      socket.disconnect();
+    };
+  }, [userId, projectId]);

-    return () => clearInterval(intervalId);
-  }, []);
+  return { tasks, isConnected, error };
+ };
+
+ import React from 'react';
+ import { useActiveTasks } from './hooks/useActiveTasks';
+
+ const ActiveTasksBoard = ({ currentUser, currentProject }) => {
+   const { tasks, isConnected, error } = useActiveTasks(currentUser.id, currentProject.id);

   return (
-    <div className="tasks-container">
-      <h3>Активні завдання</h3>
-      {tasks.length === 0 ? (
-        <p>Немає активних завдань</p>
-      ) : (
-        <ul>
-          {tasks.map((task) => (
-            <li key={task.id}>
-              {task.title} - <span>{task.status}</span>
-            </li>
-          ))}
-        </ul>
```

```
41  -       )}
59  +     <div className="tasks-board">
60  +       <div className="board-header">
61  +         <h2>Активні завдання</h2>
62  +         {/* Індикатор Real-time статусу */}
63  +         <div className={`connection-status ${isConnected ? 'online' : 'offline'}`}>
64  +           <span className="status-dot"></span>
65  +           {isConnected ? 'Оновлюється наживо' : 'Офлайн'}
66  +         </div>
67  +       </div>
68  +
69  +       {/* Відображення помилок мережі, якщо вони є */}
70  +       {error && <div className="error-banner">{error}</div>}
71  +
72  +       <div className="tasks-list-container">
73  +         {tasks.length === 0 ? (
74  +           <p className="empty-state">У цьому спринті поки немає активних завдань.</p>
75  +         ) : (
76  +           <ul className="task-list">
77  +             {tasks.map((task) => (
78  +               <li key={task.id} className="task-item">
79  +                 <span className="task-key">{task.ticketNumber}</span>
80  +                 <span className="task-title">{task.title}</span>
81  +                 <span className={`task-status status-${task.status.toLowerCase()}`}>
82  +                   {task.status}
83  +                 </span>
84  +               </li>
85  +             ))}
86  +           </ul>
87  +         )}
88  +       </div>
42  89      </div>
43  90    );
44  91  };
45  92
46  -  export default ActiveTasksList;
93  +  export default ActiveTasksBoard;
```



5. Виконано Merge pull request на GitHub



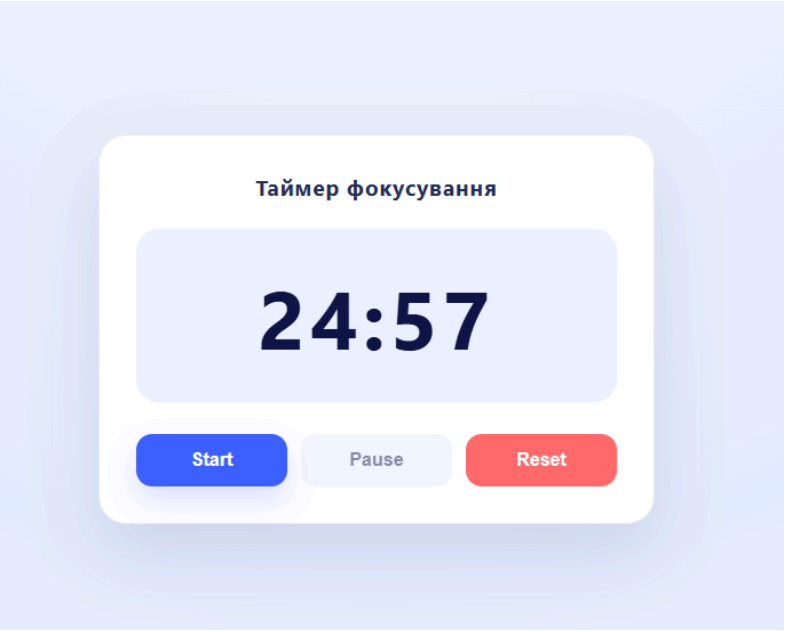
6.1. Текст промпту: Напиши код для React-компонента «Таймер фокусування» (Focus Timer), використовуючи функціональні компоненти та хуки (useState, useEffect).

Функціональні вимоги:
Великий цифровий дисплей, який відображає час у форматі XX:XX (хвилини та секунди). За замовчуванням час встановлено на 25 хвилин (1500 секунд).

Три кнопки керування:
Start (Запуск): починає зворотний відлік. Якщо таймер уже йде, кнопка має бути неактивною або прихованою.
Pause (Пауза): зупиняє таймер на поточному часі.
Reset (Скидання): скидає таймер до початкових 25 хвилин і зупиняє відлік.

Технічні вимоги:
Логіка таймера має бути оптимізована, щоб уникнути витоків пам'яті (правильне очищення setInterval при розмонтуванні компонента).

Додай базову стилізацію (можна використовувати звичайний CSS або Tailwind CSS), щоб інтерфейс виглядав сучасно, мінімалістично та мав зрозумілі акценти на кнопках керування.



6.2. Результат виконання